WIE

Diary & Highlights

Complete API Documentation for React Native Developers

Base URL: http://localhost:5010/api/diary • All routes require Bearer token • v1.0

Table of Contents

01 Service Overview & Base URL

The Diary service is part of the WIE Media Service running on port 5010. A Diary (also called a Highlight) is a named, persistent collection of Flux stories that lives on the user's profile even after the individual stories expire. Think of it as a curated album built from stories.

Service Config

```
Base URL  (development)  :  http://localhost:5010/api/diary
Base URL  (Android emu)  :  http://10.0.2.2:5010/api/diary

All diary endpoints share the prefix:  /api/diary
```

▸ What is a Diary?

A Diary is a collection owned by one user. It has:

- A title and optional cover image (uploaded to Cloudinary)
- A visibility setting (public, followers, close_friends, only_me)
- An ordered list of flux references — each flux keeps its original media URL
- An optional isPinned flag that moves it to the top of the profile highlights row
- Diary-level settings that override global defaults (allowComments, allowSharing, etc.)

> KEY CONCEPT — Diaries do not store the media themselves. They store references (fluxId + mediaUrl) to fluxes. The original flux can expire or be deleted without removing the diary entry — the mediaUrl is copied at the time the flux is added.

02 Authentication

Every endpoint in the diary service requires a valid Bearer token. The token is obtained at login and must be sent with every request via the Authorization header.

TypeScript

```
Authorization: Bearer <your_jwt_token>

// React Native — Axios instance (add to src/services/apiClients.ts)
import AsyncStorage from '@react-native-async-storage/async-storage';
import axios from 'axios';

const diaryApi = axios.create({
  baseURL: 'http://10.0.2.2:5010/api/diary',
  timeout: 20000,
});

diaryApi.interceptors.request.use(async (config) => {
  const token = await AsyncStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

A 401 Unauthorized response means the token is missing or expired. Clear the token from AsyncStorage and redirect the user to the Login screen.

03 Diary Data Model

The following fields are returned in every Diary object from the API. Understanding this shape helps you type your state correctly in React Native.

Field	Type	Required	Description
`_id`	`string`	—	Unique diary identifier (MongoDB ObjectId as string).
`userId`	`string`	—	Owner's user ID.
`title`	`string`	—	Display title of the diary / highlight.
`coverImage`	`string?`	—	Cloudinary URL of the cover image. Null if not set.
`visibility`	`string`	—	public | followers | close_friends | only_me
`fluxes`	`DiaryFlux[]`	—	Ordered array of flux references (see sub-type below).
`fluxCount`	`number`	—	Convenience count of fluxes in this diary.
`isPinned`	`boolean`	—	If true, diary appears at top of profile highlights.
`isDeleted`	`boolean`	—	Soft-delete flag. Deleted diaries are excluded from queries.
`createdAt`	`string`	—	ISO 8601 creation timestamp.
`updatedAt`	`string`	—	ISO 8601 last-update timestamp.

▸ DiaryFlux Sub-Type (each item inside fluxes[])

Field	Type	Required	Description
`fluxId`	`string`	—	Reference to the original flux _id.
`mediaUrl`	`string`	—	Media URL (copied from flux at time of adding).
`mediaType`	`string`	—	image or video.
`caption`	`string?`	—	Caption copied from the original flux.
`addedAt`	`string`	—	ISO 8601 timestamp when this flux was added to the diary.
`musicTitle`	`string?`	—	Music track title (if flux had music).
`musicArtist`	`string?`	—	Music artist name.
`musicPreviewUrl`	`string?`	—	30-second preview audio URL.
`locationLabel`	`string?`	—	Location sticker label text.

▸ Visibility Values

Value	Description
`public`	Anyone can view this diary, even non-followers.
`followers`	Only followers of the owner can view this diary.
`close_friends`	Only users in the owner's close-friends list can view.

Value	Description
`only_me`	Private — only the owner can view.

04 Diary Settings

Each user has a single DiarySettings document that controls default behaviour for all their diaries. These are global defaults — individual diaries may override some of them.

Method	Endpoint	Auth	Description
GET	/settings	✅ Yes	Fetch the current user's diary settings.
PATCH	/settings	✅ Yes	Update one or more diary settings fields.

▸ GET /diary/settings — Full Response

JSON Response

```
{
  "success": true,
  "data": {

    // ── Visibility defaults ──────────────────────────────────
    "defaultVisibility": "followers",
    // Options: 'public' | 'followers' | 'close_friends' | 'only_me'

    // ── Interaction controls ─────────────────────────────────
    "allowComments":  true,
    "allowReactions": true,
    "allowSharing":   true,

    // ── Display preferences ──────────────────────────────────
    "showFluxCount":  true,
    "autoArchive":    true,
    // autoArchive: if true, expired fluxes added to a diary
    // are kept in the diary even after expiry

    // ── Timestamps ───────────────────────────────────────────
    "createdAt": "2025-11-01T00:00:00.000Z",
    "updatedAt": "2025-12-01T10:00:00.000Z"
  }
}
```

▸ PATCH /diary/settings — Request Body

Send only the fields you want to change. All fields are optional. The server merges your changes with the existing settings document.

Field	Type	Required	Description
defaultVisibility	string	No	Default visibility for new diaries: public | followers | close_friends | only_me
allowComments	boolean	No	Allow others to comment on diary entries.
allowReactions	boolean	No	Allow emoji reactions on diary entries.

Field	Type	Required	Description
`allowSharing`	`boolean`	No	Allow others to share diaries.
`showFluxCount`	`boolean`	No	Show the number of fluxes on the diary card in profile.
`autoArchive`	`boolean`	No	Keep expired flux media inside diaries even after the flux expires.

▸ PATCH /diary/settings — Example Calls

TypeScript

```
// Disable comments on all diaries
await diaryApi.patch('/settings', { allowComments: false });

// Set default visibility to close friends only
await diaryApi.patch('/settings', { defaultVisibility: 'close_friends' });

// Multiple fields at once
await diaryApi.patch('/settings', {
  defaultVisibility: 'followers',
  allowSharing: false,
  showFluxCount: true,
});
```

> The PATCH endpoint uses partial updates — you never need to send the full settings object. Only the fields you include will be changed.

05 Create Diary

Creates a new diary for the authenticated user. Accepts multipart/form-data so the cover image can be uploaded in the same request. The cover image is uploaded to Cloudinary and stored as a URL.

Method	Endpoint	Auth	Description
POST	/create	✅ Yes	Create a new diary. Accepts multipart/form-data with optional cover image.

▸ Request Fields — multipart/form-data

Field	Type	Required	Description
title	string	Yes	Display name of the diary / highlight collection.
visibility	string	No	public | followers | close_friends | only_me. Defaults to user's defaultVisibility setting.
cover	File	No	Cover image file (jpg / png / webp). Uploaded to Cloudinary.
fluxIds	JSON[]	No	Stringified JSON array of flux IDs to pre-populate the diary.

▸ Success Response (HTTP 201)

JSON Response

```
{
  "success": true,
  "data": {
    "_id":         "diary_abc123",
    "userId":      "user_xyz",
    "title":       "Travel 2025",
    "coverImage": "https://res.cloudinary.com/.../cover.jpg",
    "visibility": "followers",
    "fluxes":      [],
    "fluxCount":   0,
    "isPinned":    false,
    "isDeleted":   false,
    "createdAt":   "2025-12-01T10:00:00.000Z",
    "updatedAt":   "2025-12-01T10:00:00.000Z"
  }
}
```

▸ React Native — Create Diary with Cover Image

TypeScript

```
import { launchImageLibrary } from 'react-native-image-picker';

const createDiary = async () => {
```

```
  // 1. Pick cover image (optional)
  const result = await launchImageLibrary({ mediaType: 'photo', quality: 0.85 });
  const asset  = result.assets?.[0];

  const formData = new FormData();
  formData.append('title', 'Travel 2025');
  formData.append('visibility', 'followers');

  // 2. Attach cover image if selected
  // ☑ React Native uses { uri, name, type } — NOT File / Blob
  if (asset) {
    formData.append('cover', {
      uri:  asset.uri,
      name: asset.fileName || 'cover.jpg',
      type: asset.type   || 'image/jpeg',
    } as any);
  }

  // 3. Optional: pre-populate with existing flux IDs
  formData.append('fluxIds', JSON.stringify(['flux_001', 'flux_002']));

  const res = await diaryApi.post('/create', formData, {
    headers: { 'Content-Type': 'multipart/form-data' },
  });

  return res.data.data; // Diary object
};
```

06 Get User Diaries

Returns all diaries owned by a specific user. This is used to populate the Highlights row on both the self-profile and other users' profiles. The server automatically enforces visibility — a viewer cannot see a diary set to only_me unless they are the owner, or close_friends unless they are in the owner's close-friends list.

Method	Endpoint	Auth	Description
GET	/user/:userId	✅ Yes	Get all diaries for a user (visibility-filtered for the viewer).

▸ URL Parameter

Parameter	Description
:userId	The user ID of the profile being viewed. Use your own ID on self-profile.

▸ Success Response (HTTP 200)

JSON Response

```
{
  "success": true,
  "data": [
    {
      "_id":         "diary_abc123",
      "userId":      "user_xyz",
      "title":       "Travel 2025",
      "coverImage": "https://res.cloudinary.com/.../cover.jpg",
      "visibility": "followers",
      "fluxes": [
        {
          "fluxId":    "flux_001",
          "mediaUrl":  "https://res.cloudinary.com/.../story.jpg",
          "mediaType": "image",
          "addedAt":   "2025-12-01T10:30:00.000Z"
        }
      ],
      "fluxCount": 12,
      "isPinned":  true,
      "createdAt": "2025-11-15T08:00:00.000Z"
    }
  ]
}
```

▸ Rendering Highlights Row — React Native

TypeScript

```
// Pinned diaries should always appear first
```

```
const sorted = diaries.sort((a, b) => {
  if (a.isPinned && !b.isPinned) return -1;
  if (!a.isPinned && b.isPinned) return  1;
  return new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime();
});

// Cover image fallback: if no coverImage, use first flux mediaUrl
const getCover = (diary) =>
  diary.coverImage || diary.fluxes?.[0]?.mediaUrl || null;
```

07 Get Single Diary

Returns the full diary document including all its flux entries. Access is controlled by the diary's visibility setting and the viewer's relationship with the owner.

Method	Endpoint	Auth	Description
GET	/:diaryId	✅ Yes	Get a single diary by ID (visibility-enforced).

▸ URL Parameter

Parameter	Description
:diaryId	The _id of the diary to fetch.

▸ Success Response (HTTP 200)

JSON Response

```
{
  "success": true,
  "data": {
    "_id":         "diary_abc123",
    "title":       "Travel 2025",
    "coverImage": "https://res.cloudinary.com/.../cover.jpg",
    "visibility": "followers",
    "fluxes": [
      {
        "fluxId":          "flux_001",
        "mediaUrl":        "https://res.cloudinary.com/.../story.mp4",
        "mediaType":       "video",
        "caption":         "At the beach!",
        "addedAt":         "2025-12-01T10:30:00.000Z",
        "musicTitle":      "Shape of You",
        "musicArtist":     "Ed Sheeran",
        "musicPreviewUrl": "https://audio-ssl.itunes.apple.com/.../preview.m4a",
        "locationLabel":   "Sydney Opera House"
      }
    ],
    "fluxCount": 12,
    "isPinned":  true,
    "isDeleted": false,
    "createdAt": "2025-11-15T08:00:00.000Z",
    "updatedAt": "2025-12-01T10:30:00.000Z"
  }
}
```

▸ Error Responses

Status	Meaning
`404 Not Found`	Diary does not exist, has been deleted, or the viewer does not have access.
`401 Unauthorized`	Missing or invalid Bearer token.

08 Edit Diary

Allows the owner to update a diary's title, visibility, and/or cover image. Uses multipart/form-data so the cover image can be changed in the same request. Only the diary owner can edit.

Method	Endpoint	Auth	Description
PATCH	/:diaryId	✅ Yes	Update diary title, visibility, or cover image. Owner only.

▸ Request Fields — multipart/form-data

Field	Type	Required	Description
title	string	No	New display title for the diary.
visibility	string	No	New visibility: public | followers | close_friends | only_me
cover	File	No	New cover image file. Replaces the existing cover on Cloudinary.

▸ Success Response (HTTP 200)

JSON Response

```
{
  "success": true,
  "data": { /* Updated Diary object */ }
}
```

▸ React Native — Edit with New Cover

TypeScript

```
const editDiary = async (diaryId: string) => {
  const formData = new FormData();
  formData.append('title',      'Summer Memories');
  formData.append('visibility', 'public');

  if (newCoverAsset) {
    formData.append('cover', {
      uri:  newCoverAsset.uri,
      name: newCoverAsset.fileName || 'cover.jpg',
      type: newCoverAsset.type     || 'image/jpeg',
    } as any);
  }

  const res = await diaryApi.patch(`/${diaryId}`, formData, {
    headers: { 'Content-Type': 'multipart/form-data' },
  });
  return res.data.data;
};
```

09 Delete Diary

Soft-deletes a diary. The diary is marked as isDeleted=true and excluded from all queries. The underlying flux media is NOT deleted — only the diary container is removed. Only the diary owner can delete.

Method	Endpoint	Auth	Description
DELETE	/:diaryId	✅ Yes	Soft-delete a diary. Owner only. Does not delete the flux media.

▸ Success Response (HTTP 200)

JSON Response

```
{ "success": true, "message": "Diary deleted" }
```

▸ Error Responses

Status	Meaning
403 Forbidden	Caller is not the owner of this diary.
404 Not Found	Diary does not exist or is already deleted.

> Deleting a diary is reversible only at the database level. Consider showing a confirmation dialog in the UI before calling this endpoint.

10 Highlight a Flux (Smart Add)

This is the primary smart endpoint for adding a story to highlights. It handles three scenarios automatically without any extra logic from the client.

Scenario	Server Behaviour
`Scenario 1 — diaryId provided`	Adds the flux to an existing diary. Returns action: 'added'.
`Scenario 2 — newDiaryTitle provided`	Creates a brand-new diary with that title, then adds the flux. Returns action: 'created'.
`Scenario 3 — neither diaryId nor title`	Returns action: 'pick' along with the user's existing diaries so the client can show a picker UI.

Method	Endpoint	Auth	Description
`POST`	`/highlight`	✅ Yes	Smart-add a flux to highlights. Handles existing diary, new diary, or picker.

▸ Request Body — application/json

Field	Type	Required	Description
`fluxId`	`string`	Yes	The _id of the flux to highlight.
`diaryId`	`string`	No	Existing diary _id to add the flux to.
`newDiaryTitle`	`string`	No	If provided, creates a new diary with this title and adds the flux.

▸ Response — Scenario 1: Added to existing diary (HTTP 200)

JSON Response

```
{
  "success": true,
  "action": "added",
  "data": { /* Updated Diary object */ }
}
```

▸ Response — Scenario 2: New diary created (HTTP 201)

JSON Response

```
{
  "success": true,
  "action": "created",
  "data": { /* Newly created Diary object */ }
}
```

▸ Response — Scenario 3: User must pick a diary (HTTP 200)

JSON Response

```
{
  "success": true,
  "action": "pick",
  "diaries": [
    { "_id": "diary_001", "title": "Travel 2025", "coverImage": "..." },
    { "_id": "diary_002", "title": "Friends",      "coverImage": "..." }
  ]
}
```

▸ React Native — Handle All Three Scenarios

TypeScript

```
const highlightFlux = async (fluxId: string, diaryId?: string, newTitle?: string)
=> {
  const body: Record<string, string> = { fluxId };
  if (diaryId)   body.diaryId      = diaryId;
  if (newTitle)  body.newDiaryTitle = newTitle;

  const res = await diaryApi.post('/highlight', body);
  const { action, data, diaries } = res.data;

  if (action === 'added' || action === 'created') {
    // Success — show toast 'Added to highlights'
    showToast('Added to highlights ✓');
    return data; // Diary object
  }

  if (action === 'pick') {
    // Show a bottom sheet listing the user's diaries
    setDiaryPickerVisible(true);
    setDiaryPickerOptions(diaries);
    setPendingFluxId(fluxId);
  }
};

// When user picks from the bottom sheet:
const onDiaryPicked = (pickedDiaryId: string) => {
  highlightFlux(pendingFluxId, pickedDiaryId);
  setDiaryPickerVisible(false);
};
```

11 Add Flux to Diary

Directly adds a flux to a specific diary. Unlike the highlight endpoint, this requires you to know the exact diaryId. Use this when the user has already chosen a diary (e.g. from a picker screen).

Method	Endpoint	Auth	Description
POST	/:diaryId/add-flux	✅ Yes	Add a flux to an existing diary. Owner only.

▸ Request Body — application/json

Field	Type	Required	Description
fluxId	string	Yes	The _id of the flux to add to this diary.

▸ Success Response (HTTP 200)

JSON Response

```
{
  "success": true,
  "data": { /* Updated Diary object with new flux appended to fluxes[] */ }
}
```

▸ Error Responses

Status	Meaning
400 Bad Request	fluxId is missing, flux not found, or duplicate flux in diary.
403 Forbidden	Caller is not the owner of this diary.

12 Remove Flux from Diary

Removes a specific flux reference from a diary. The flux itself is not deleted — only its entry in the diary's fluxes array is removed. Only the diary owner can remove fluxes.

Method	Endpoint	Auth	Description
DELETE	/:diaryId/flux/:fluxId	✅ Yes	Remove a flux from a diary. Owner only.

▸ URL Parameters

Parameter	Description
:diaryId	The _id of the diary.
:fluxId	The fluxId of the entry to remove from the diary's fluxes[].

▸ Success Response (HTTP 200)

JSON Response

```
{
  "success": true,
  "data": { /* Updated Diary object with flux removed from fluxes[] */ }
}
```

▸ Error Responses

Status	Meaning
400 Bad Request	Flux not found in this diary.
403 Forbidden	Caller is not the owner of this diary.

13 Reorder Fluxes inside a Diary

Allows the owner to drag-and-drop reorder the fluxes inside a diary. Send the complete ordered array of fluxId strings in the new desired order. The server replaces the fluxes ordering entirely.

Method	Endpoint	Auth	Description
PATCH	/:diaryId/reorder	✅ Yes	Reorder fluxes inside a diary. Send full ordered fluxId array. Owner only.

▸ Request Body — application/json

Field	Type	Required	Description
orderedFluxIds	string[]	Yes	Complete ordered array of fluxId strings representing the new order.

JSON Body

```
// Body example — full array of flux IDs in the new desired order
{
  "orderedFluxIds": ["flux_003", "flux_001", "flux_002"]
}
```

▸ Success Response (HTTP 200)

JSON Response

```
{
  "success": true,
  "data": { /* Diary object with fluxes[] in the new order */ }
}
```

▸ React Native — Drag-and-Drop Reorder

TypeScript

```
// Use react-native-draggable-flatlist or similar
import DraggableFlatList from 'react-native-draggable-flatlist';

const onDragEnd = async ({ data: reorderedFluxes }) => {
  setFluxes(reorderedFluxes); // optimistic update

  const orderedFluxIds = reorderedFluxes.map(f => f.fluxId);

  try {
    await diaryApi.patch(`/${diaryId}/reorder`, { orderedFluxIds });
  } catch {
    setFluxes(originalFluxes); // roll back on error
    Alert.alert('Reorder failed', 'Please try again.');
  }
};
```

14 Pin & Unpin Diary

Toggles the isPinned flag on a diary. Pinned diaries appear at the top of the profile highlights row. Calling this endpoint again on a pinned diary unpins it. Only the owner can pin/unpin.

Method	Endpoint	Auth	Description
PATCH	/:diaryId/pin	✅ Yes	Toggle pin/unpin for a diary. Owner only.

▸ Success Response (HTTP 200)

JSON Response

```
{
  "success":  true,
  "isPinned": true,
  "data": { /* Updated Diary object */ }
}
```

The isPinned field in the top-level response tells you the new state. You do not need to read it from data.isPinned — both are available.

> There is no server-enforced limit on the number of pinned diaries. However, it is recommended to show only the first 1–3 pinned diaries prominently in the UI and let the rest scroll horizontally.

15 Full Route Reference

All routes are relative to http://localhost:5010/api/diary. Static routes (without :diaryId) must come before parameterised routes in the router — this matches the server implementation.

▸ Static Routes — must be before /:diaryId

Method	Endpoint	Auth	Description
GET	/settings	✅ Yes	Get current user's diary settings
PATCH	/settings	✅ Yes	Update diary settings (partial update)
POST	/create	✅ Yes	Create a new diary (multipart/form-data)
GET	/user/:userId	✅ Yes	Get all diaries for a user (visibility-filtered)
POST	/highlight	✅ Yes	Smart-add flux to highlights (add / create / pick)

▸ Parameterised Routes — /:diaryId

Method	Endpoint	Auth	Description
GET	/:diaryId	✅ Yes	Get a single diary by ID
PATCH	/:diaryId	✅ Yes	Edit diary title / visibility / cover
DELETE	/:diaryId	✅ Yes	Soft-delete a diary
POST	/:diaryId/add-flux	✅ Yes	Add a flux to a specific diary
DELETE	/:diaryId/flux/:fluxId	✅ Yes	Remove a flux from a diary
PATCH	/:diaryId/reorder	✅ Yes	Reorder fluxes inside a diary
PATCH	/:diaryId/pin	✅ Yes	Toggle pin/unpin on a diary

16 React Native Integration Examples

▸ Complete diaryService.ts for React Native

TypeScript

```
// src/services/diaryService.ts
import AsyncStorage from '@react-native-async-storage/async-storage';
import axios from 'axios';
import Config from 'react-native-config';

const BASE = (Config.MEDIA_API_URL || 'http://10.0.2.2:5010/api') + '/diary';

const diaryApi = axios.create({ baseURL: BASE, timeout: 20000 });

diaryApi.interceptors.request.use(async (cfg) => {
  const token = await AsyncStorage.getItem('token');
  if (token) cfg.headers.Authorization = `Bearer ${token}`;
  return cfg;
});

// ── Settings ──────────────────────────────────────────────────
export const getDiarySettings = () =>
  diaryApi.get('/settings').then(r => r.data.data);

export const updateDiarySettings = (patch: Record<string, any>) =>
  diaryApi.patch('/settings', patch).then(r => r.data.data);

// ── CRUD ──────────────────────────────────────────────────────
export const getUserDiaries = (userId: string) =>
  diaryApi.get(`/user/${userId}`).then(r => r.data.data);

export const getDiaryById = (diaryId: string) =>
  diaryApi.get(`/${diaryId}`).then(r => r.data.data);

export const deleteDiary = (diaryId: string) =>
  diaryApi.delete(`/${diaryId}`);

// ── Flux management ───────────────────────────────────────────
export const highlightFlux = (body: {
  fluxId: string;
  diaryId?: string;
  newDiaryTitle?: string;
}) => diaryApi.post('/highlight', body).then(r => r.data);

export const addFluxToDiary = (diaryId: string, fluxId: string) =>
  diaryApi.post(`/${diaryId}/add-flux`, { fluxId }).then(r => r.data.data);

export const removeFluxFromDiary = (diaryId: string, fluxId: string) =>
  diaryApi.delete(`/${diaryId}/flux/${fluxId}`).then(r => r.data.data);

export const reorderDiaryFluxes = (diaryId: string, orderedFluxIds: string[]) =>
  diaryApi.patch(`/${diaryId}/reorder`, { orderedFluxIds }).then(r => r.data.data);

export const togglePinDiary = (diaryId: string) =>
```

```
  diaryApi.patch(`/${diaryId}/pin`).then(r => r.data);
```

▸ Profile Screen — Loading Highlights Row

TypeScript

```
// src/screens/ProfileScreen.tsx (highlights section)

const [diaries, setDiaries] = useState([]);

useEffect(() => {
  if (userId) loadDiaries();
}, [userId]);

const loadDiaries = async () => {
  try {
    const data = await getUserDiaries(userId);
    // Pinned diaries always come first
    const sorted = [...data].sort((a, b) => {
      if (a.isPinned && !b.isPinned) return -1;
      if (!a.isPinned && b.isPinned) return  1;
      return 0;
    });
    setDiaries(sorted);
  } catch (err) {
    console.error('Failed to load diaries:', err);
  }
};

// Cover image fallback helper
const getCoverUri = (diary) =>
  diary.coverImage || diary.fluxes?.[0]?.mediaUrl || null;
```

▸ Add to Highlights — Full UI Flow

TypeScript

```
// When user taps 'Add to Highlight' on a flux:
const onAddToHighlight = async (fluxId: string) => {

  // Step 1: Call highlight endpoint with NO diaryId or title
  const result = await highlightFlux({ fluxId });

  if (result.action === 'added' || result.action === 'created') {
    // Done — show success toast
    showToast('Added to Highlights ✓');
    loadDiaries(); // refresh highlights row
    return;
  }

  if (result.action === 'pick') {
    // Step 2: Show a bottom sheet with existing diaries + 'Create New' option
    showDiaryPicker({
      diaries: result.diaries,
      onPickDiary: async (diaryId) => {
```

```
        await highlightFlux({ fluxId, diaryId });
        showToast('Added to Highlights ✓');
        loadDiaries();
      },
      onCreateNew: async (title) => {
        await highlightFlux({ fluxId, newDiaryTitle: title });
        showToast('New Highlight created ✓');
        loadDiaries();
      },
    });
  }
};
```